



PUSL3190 Computing Project

Project Interim Report

Stock Price Indicator and Prediction Project

Supervisor: MR. Diluka Wijesinghe

Name: Wakishta W Dissanayake

Plymouth Index Number: 10953506

Degree Program: BSc.(Hons) Computer science

Table of Contents

Chapter 01: Introduction	4
Introduction	4
Problem Definition.....	4
Purpose and Scope.....	5
Chapter 02: System Analysis	6
2.1 Facts Gathering Techniques	6
2.2 Existing System	6
2.3 Drawbacks of the Existing System	6
Chapter 03: Requirements Specification	7
3.1 Functional Requirements	7
3.2 Non-Functional Requirements	7
3.3 Hardware / Software Requirements.....	7
Chapter 04: Literature Review.....	8
4.1 Introduction and Search Strategy.....	8
4.2 Thematic Organization	8
4.3 Critical Analysis and Identified Gaps.....	9
4.4 Implications for Design Choices.....	9
Feasibility Study.....	9
4.5 Operational Feasibility.....	9
4.6 Economical Feasibility	10
4.7 Technical Feasibility.....	10
Chapter 05: System Architecture	11
5.1 Use Case Diagram.....	11
5.4 High-level Architectural Diagram	11
Chapter 06 – Development Tools and Technologies	13
6.1 Development Methodology	13
6.2 Programming Languages and Tools	13
6.3 Third-Party Components and Libraries	14
Chapter 07: Implementation Progress	15
7.1 Development Environment Setup	15
7.2 Implemented Features	15
7.3 Challenges Encountered and Solutions.....	15
Chapter 08: Discussion	16
8.1 Summary of the Report	16

8.2 What has changed from the Proposal.....	16
8.3 Future Plans	16
Chapter 09: Referencing.....	17
Appendices	17
Appendix A: User Requirements Research Survey	17
A.1 Research Methodology and Purpose.....	17
A.2 Survey Instrument Structure	17
Section 1: Demographics and Financial Literacy	18
Section 2: Current Tool Analysis	18
Section 3: Requirement Validation (Non-Functional).....	18
Section 4: Feature Prioritization	18
A.3 Survey Screenshots	19

Chapter 01: Introduction

Introduction

The modern financial landscape is characterized by extreme volatility and an overwhelming influx of data, making traditional market analysis increasingly difficult for the average participant. While institutional investors utilize high-frequency data and enterprise-grade AI, individual investors often rely on lagging indicators or manual observations. This project sits at the intersection of Financial Technology (FinTech) and Deep Learning, specifically focusing on the application of Long Short-Term Memory (LSTM) networks.

Stock prices are inherently time-series data, meaning their future values are influenced by historical sequences. Traditional models often fail because they treat these relationships as linear; however, deep learning models like LSTMs are designed to capture "long-term dependencies"-the ability to remember and relate events from far in the past to the present moment. By transitioning these models from isolated research environments into a production-ready Next.js web application, this project seeks to democratize access to advanced predictive insights.

Problem Definition

The stock market is a highly dynamic and unpredictable environment where behavior is influenced by numerous factors that interact in complex, non-linear ways.

The Technical Gap: Traditional forecasting methods, such as ARIMA or simple Linear Regression, often struggle to capture these intricate patterns, leading to unreliable predictions.

The Accessibility Gap: In recent years, Deep Learning models have shown success, but they are typically limited to specialized Python environments like Jupyter notebooks, which are not accessible to everyday users.

The Cost Barrier: Individual investors and decision-makers face high-cost enterprise tools that remain out of reach for small investors or independent researchers.

The User Experience Gap: Many existing forecasting systems rely on outdated models and offer poor user experiences, lacking the interactivity required for modern financial analysis.

Purpose and Scope

The primary purpose of this project is to develop a robust, end-to-end stock price forecasting application delivered through a high-performance web interface. The project encompasses the entire lifecycle from data acquisition to cloud deployment.

Data Acquisition: Collecting historical stock data from reliable APIs like Yahoo Finance, involving cleaning, normalizing, and converting data into time-series sequences.

Feature Engineering: Computing key technical indicators-including the Moving Average (MA), Relative Strength Index (RSI), and Moving Average Convergence Divergence (MACD)-to enhance the model's predictive capability.

System Architecture: Implementing a decoupled microservice architecture where the Python ML Model is an independent RESTful service (using FastAPI) callable by the Next.js frontend.

Limitations: To maintain a realistic plan, the project excludes real-time trading execution, integration with brokerage APIs, and news-based sentiment forecasting.

Chapter 02: System Analysis

2.1 Facts Gathering Techniques

To ensure the project is based on research rather than assumptions, the following techniques were used:

Literature Review: Extensive search of databases like IEEE Xplore and ResearchGate for studies comparing statistical and deep learning models.

Domain Expertise: Consultation with practicing financial analysts to validate the utility of technical indicators.

Document Analysis: Review of existing financial API documentation (Yahoo Finance) to understand data structures and constraints.

2.2 Existing System

The existing system for most independent researchers consists of Python scripts running locally in IDEs or notebooks. In the broader industry, users rely on spreadsheet-based management or expensive enterprise platforms like Bloomberg Terminals.

2.3 Drawbacks of the Existing System

Linear Limitations: Models like ARIMA are only effective for linear and stationary datasets, failing in volatile markets.

Monolithic Setup: Current implementations are often not designed for scalable web-based accessibility.

High Latency: Lack of microservice decoupling leads to poor performance when running heavy computations alongside a UI.

Chapter 03: Requirements Specification

3.1 Functional Requirements

FR1 (Authentication): The system shall allow users to register and securely log in using JWT.

FR2 (Prediction Request): The system shall allow users to input a stock ticker and a prediction horizon (e.g., 7 days).

FR3 (Visualization): The frontend shall display historical data and projected forecasts via Chart.js.

FR4 (Microservice Call): Next.js API routes shall send features to the Python ML microservice via secure HTTP.

3.2 Non-Functional Requirements

Performance: The complete prediction workflow shall take less than 8 seconds.

Security: All user credentials and API communications must be encrypted using HTTPS and JWT.

Scalability: The application must be deployable as two separate, containerized services to facilitate independent scaling.

3.3 Hardware / Software Requirements

Software: Python 3.x, TensorFlow/Keras, FastAPI, Next.js, Tailwind CSS, and MongoDB.

Hardware: High-performance PC with minimum 16GB RAM for model training.

Chapter 04: Literature Review

4.1 Introduction and Search Strategy

The purpose of this review is to evaluate the progression of time-series forecasting from statistical foundations to modern deep learning implementations. The search strategy focused on identifying studies comparing statistical and deep learning models for financial data. Keywords used include "LSTM," "ARIMA," "Stock Price Prediction," and "Microservice Architecture". Databases such as IEEE Xplore, ResearchGate, and SpringerLink were utilized to find scholarly articles published between 2014 and 2025.

4.2 Thematic Organization

A. Traditional Statistical Models (ARIMA)

Early research in stock forecasting heavily utilized the Autoregressive Integrated Moving Average (ARIMA) model. Studies like Ariyo et al. (2014) demonstrate that while ARIMA is effective for linear and stationary datasets, it struggles with the high volatility and non-linear nature of real-world financial markets. ARIMA models essentially assume that past values have a linear relationship with future values, which fails during market shocks or rapid trend shifts.

B. Deep Learning and LSTM Networks

Recent literature shows a significant shift toward Long Short-Term Memory (LSTM) networks to capture long-term dependencies in time-series data.

Non-linear Processing: Wu, Chen, and Ding (2023) and Yu (2025) highlight that LSTMs outperform traditional models by handling temporal relationships and dynamic patterns more effectively.

Multivariate Indicators: Integrating technical indicators like RSI and MACD into these models further enhances predictive accuracy by providing the model with more context than price alone.

Comparative Advantage: Li et al. (2023) demonstrated that LSTMs specifically excel in predicting technology stocks where volatility is high and traditional linear regressions fail to find support levels.

C. System Deployment and Accessibility

Most existing studies focus on the algorithmic performance within isolated Python environments like Jupyter notebooks. There is a noted lack of research on the practical deployment of these models as scalable web services. Current solutions are often monolithic and not designed for web-based accessibility, making them difficult for everyday investors to use.

4.3 Critical Analysis and Identified Gaps

Methodological Gap: While LSTMs are powerful, many implementations lack a direct comparison with conventional models like ARIMA within a single production-ready interface.

Architectural Gap: There is a clear absence of microservice-based solutions that decouple heavy machine learning computation (Python) from responsive user interfaces (Next.js).

Accessibility Gap: Advanced predictive tools are currently locked behind high-cost enterprise paywalls or require significant programming knowledge to operate.

4.4 Implications for Design Choices

This project addresses these gaps by implementing a decoupled microservice architecture. By deploying the LSTM model via a FastAPI backend and a Next.js frontend, the system will provide high-performance forecasting within a scalable, user-friendly platform. This ensures that the computational load of the model does not impact the responsiveness of the web application layer.

Feasibility Study

4.5 Operational Feasibility

The project addresses a critical lack of accessibility in the FinTech space by providing professional-grade forecasting to independent researchers. The web interface will prioritize clarity and speed to encourage adoption by users with different levels of expertise.

4.6 Economical Feasibility

The use of open-source technologies such as React and TensorFlow significantly reduces development costs. The total estimated cost for hosting and domain services is 30,000 LKR.

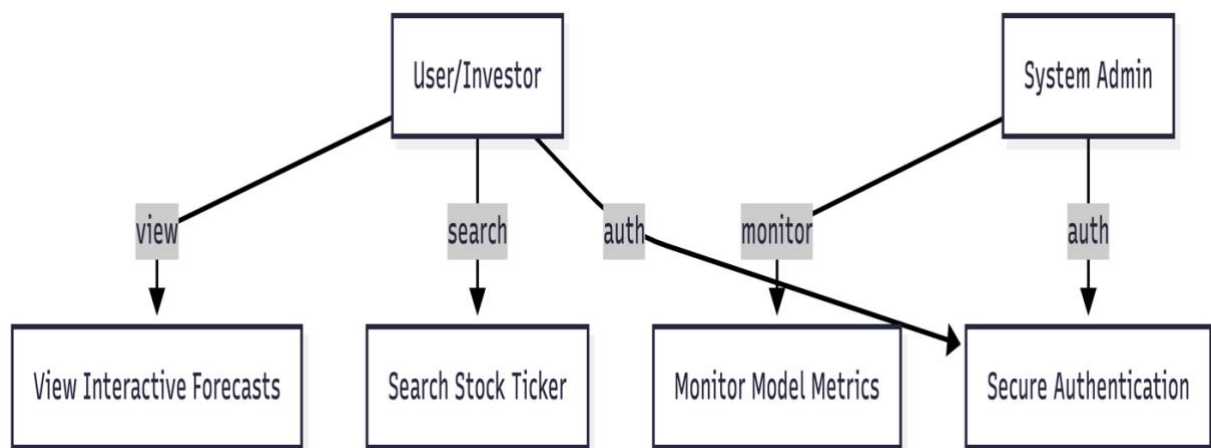
4.7 Technical Feasibility

The development team has prior experience in full-stack development. LSTMs are scientifically proven to handle the non-linear relationships of real-world financial data better than traditional models.

Chapter 05: System Architecture

5.1 Use Case Diagram

The system supports multiple actors, including the User/Investor who interacts with the frontend to search for tickers and view forecasts. Admin roles oversee system maintenance and performance monitoring.



5.4 High-level Architectural Diagram

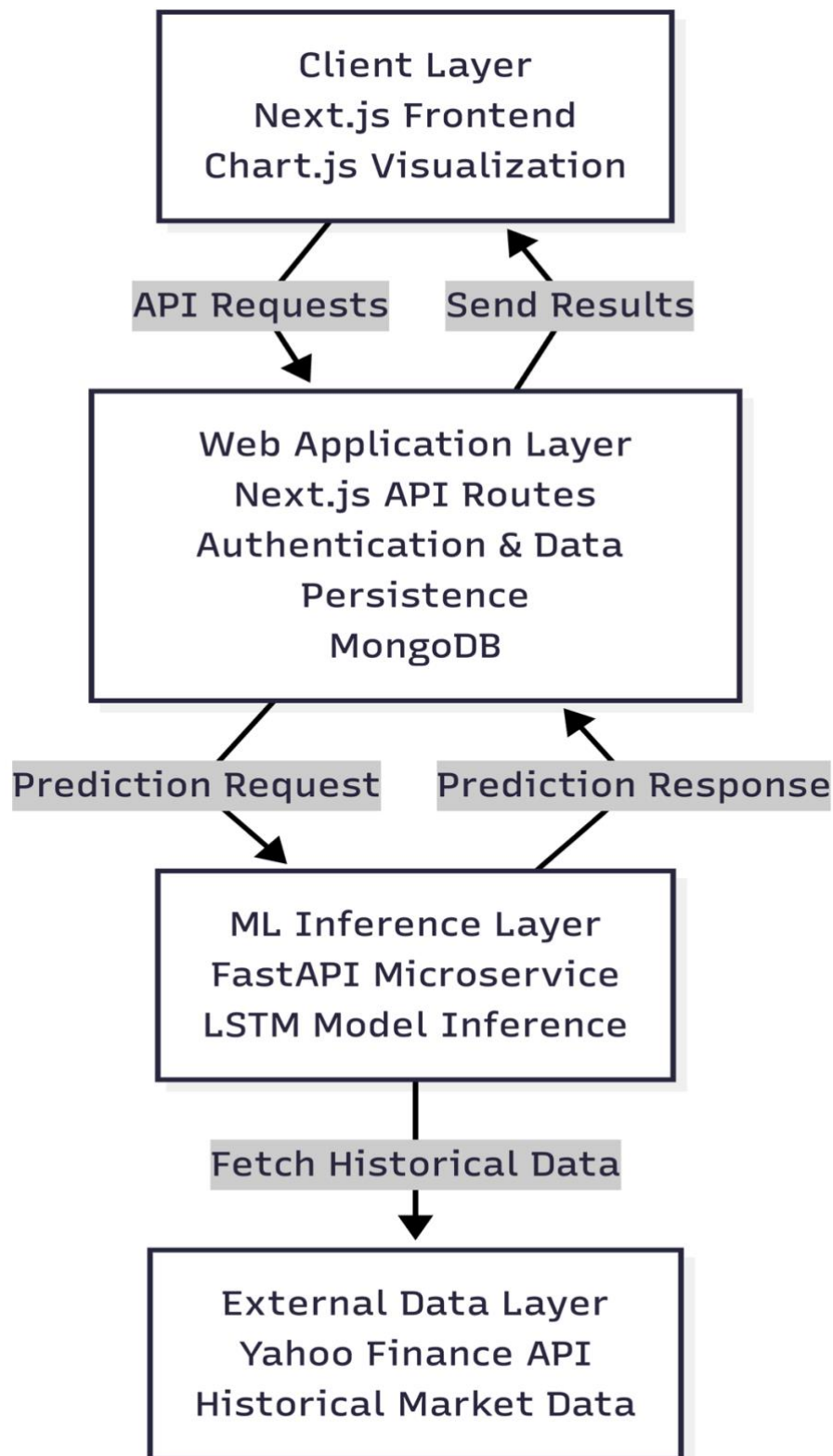
The system is designed around a decoupled microservice strategy:

Client Layer: Responsive Next.js Frontend for user interaction.

Web Application Layer: Next.js API Routes manage authentication and data persistence using MongoDB.

ML Inference Layer: A FastAPI/Flask microservice that interacts with the trained LSTM model.

External Data Layer: Fetches real-time financial data from the Yahoo Finance API.



Chapter 06 – Development Tools and Technologies

This chapter provides a comprehensive justification for the technical stack selected to bridge the gap between academic machine learning and practical web deployment.

6.1 Development Methodology

The project adopts the Agile Development Methodology, specifically utilizing the Scrum framework. This choice is driven by the inherent complexity of integrating a Python-based machine learning microservice with a JavaScript-based web frontend.

Iterative Development: The project is divided into five distinct Sprints, allowing for continuous testing and refinement of the LSTM model before it is integrated into the final UI.

Flexibility: Financial APIs and deep learning libraries frequently undergo updates; an Agile approach allows the project to adapt to technical changes without derailing the entire timeline.

Continuous Integration: By decoupling the ML inference layer, the system can be updated with better weights (model versioning) without requiring a full redeployment of the Next.js frontend.

6.2 Programming Languages and Tools

Python 3.x: Selected as the primary language for the ML Inference Layer due to its robust ecosystem for data science and deep learning.

TensorFlow/Keras: These libraries are utilized to design, train, and optimize the multivariate LSTM network because they offer high-level APIs for managing complex time-series sequences.

Next.js (React Framework): Chosen for the Frontend Layer to provide a high-performance, responsive user experience through Server-Side Rendering (SSR) and optimized API routes.

FastAPI: Used to build the RESTful microservice. FastAPI was selected over traditional Flask because of its asynchronous capabilities, which are essential for meeting the 8-second response time requirement (NFR1).

Tailwind CSS: Employed for rapid UI development, ensuring the platform is 100% responsive for both desktop and mobile users (NFR4).

6.3 Third-Party Components and Libraries

Yahoo Finance API (yfinance): Acts as the primary data source, providing historical price data and trading volume required for model training.

Chart.js: Utilized to render interactive, real-time visualizations of both historical trends and the 7-day predicted horizon.

JWT (JSON Web Tokens): Implemented for secure, stateless authentication between the Next.js frontend and the FastAPI backend (NFR2).

Pandas & NumPy: Essential for data cleaning, normalization, and the calculation of technical indicators like RSI and MACD.

Chapter 07: Implementation Progress

7.1 Development Environment Setup

The development environment is containerized using Docker to ensure environment parity across all stages. This allows seamless transition between local development and cloud platforms like Vercel and AWS.

7.2 Implemented Features

Data Acquisition: Automated script to fetch 5–10 years of historical data from Yahoo Finance.

Preprocessing Pipeline: Implementation of Min-Max scaling and technical indicator calculations (RSI, MACD).

Model Training: Initial LSTM architecture has been designed and training on historical data has commenced.

Authentication Service: JWT-based login and registration are functional within the Next.js API routes.

7.3 Challenges Encountered and Solutions

Challenge: Model Overfitting, where the LSTM memorizes historical noise instead of general patterns.

Solution: Implemented Early Stopping, Dropout layers, and L2 Regularization during the training phase.

Challenge: API Latency in the Python microservice.

Solution: Utilizing FastAPI's asynchronous capabilities and implementing caching for frequently requested tickers.

Chapter 08: Discussion

8.1 Summary of the Report

This interim report demonstrates that the project is technically sound and proceeding according to the initial project plan. The architecture has been successfully decoupled, and the machine learning pipeline is in progress.

8.2 What has changed from the Proposal

Compared to the initial proposal, the system scope was refined to prioritize core academic functionalities. Real-time trading execution and news-based sentiment analysis were moved to future plans to ensure the high-accuracy forecasting goals are met within the project timeframe.

8.3 Future Plans

Upcoming work includes:

Full API Bridge integration between Next.js and the ML service.

Completing interactive visualization with Chart.js.

Rigorous model validation against ARIMA benchmarks to justify superiority.

Chapter 09: Referencing

Indian stock market prediction using artificial neural networks on tick data Published: 21 March 2019. Available at <https://jfin-swufe.springeropen.com/articles/10.1186/s40854-019-0131-7>

AI Stock Market Prediction with Python and LSTM." (2025). Artificial Intelligence in Plain English. Available at <https://ai.plainenglish.io/ai-powered-stock-market-prediction-with-python-and-lstm-dd7c82024bdf>

Ariyo, A. A., Adewumi, A. O., & Ayo, C. K. (2014). "Stock Price Prediction Using the ARIMA Model." In Proceedings of the 2014 UKSim-AMSS 16th International Conference on Computer Modelling and Simulation. Available at <https://ieeexplore.ieee.org/document/7046047>

Sunki, A., et al. (2024). "Time series forecasting of stock market using ARIMA, LSTM and FB prophet." MATEC Web of Conferences. Available at https://www.researchgate.net/publication/379050041_Time_series_forecasting_of_stock_market_using_ARIMA_LSTM_and_FB_prophet

Wu, H., Chen, S., & Ding, Y. (2023). "Comparison of ARIMA and LSTM for Stock Price Prediction." Financial Engineering and Risk Management. Available at <https://www.scrip.org/reference/referencespapers?referenceid=3830243>

Yu, Y. (2025). "LSTM-Based Time Series Prediction Model: A Case Study with YFinance Stock Data." ITM Web of Conferences. Available at https://www.researchgate.net/publication/388315149_LSTM-Based_Time_Series_Prediction_Model_A_Case_Study_with_YFinance_Stock_Data

Li, Z., et al. (2023). "Stock Market Analysis and Prediction Using LSTM: A Case Study on Technology Stocks." Innovations in Applied Engineering and Technology. Available at https://www.researchgate.net/publication/379811995_Stock_Market_Analysis_and_Prediction_Using_LSTM_A_Case_Study_on_Technology_Stocks

<https://ai.plainenglish.io/ai-powered-stock-market-prediction-with-python-and-lstm-dd7c82024bdf>

Appendices

Appendix A: User Requirements Research Survey

A.1 Research Methodology and Purpose

The survey was designed as a primary data collection instrument to validate the architectural and functional requirements of the proposed Stock Price Indicator and Prediction Project. The primary objective was to empirically verify the "Accessibility Gap" and "Technological Gap" identified in the preliminary literature review. By targeting a diverse demographic of individual investors and academic researchers, the survey aimed to justify the implementation of a decoupled microservice architecture and a Next.js web-based interface.

A.2 Survey Instrument Structure

The survey was administered via Google Forms and structured into four thematic sections:

Section 1: Demographics and Financial Literacy

- **Question 1.1:** How would you describe your experience level in stock market investing? (Beginner, Intermediate, Advanced / Professional).
- **Question 1.2:** What is your primary method for analyzing stocks before making a decision? (Technical Analysis, Fundamental Analysis, Intuition, Automated tools).
- **Question 1.3:** On a scale of 1-5, how familiar are you with Deep Learning concepts such as LSTM?.

Section 2: Current Tool Analysis

- **Question 2.1:** Which tools do you currently use for stock forecasting? (Excel, Standard Brokerage Apps, Python/Jupyter Notebooks, Enterprise Terminals) .
- **Question 2.2:** What is the biggest challenge you face when using advanced AI models? (High cost, coding knowledge required, poor UI, or high latency) .

Section 3: Requirement Validation (Non-Functional)

- **Question 3.1:** How important is it to access predictions via a web browser rather than a coding environment? (Scale 1-5) .
- **Question 3.2:** How critical is system speed for you (e.g., getting a prediction in under 8 seconds)? (Scale 1-5) .

Section 4: Feature Prioritization

- **Question 4.1:** Which indicators are most useful when viewing a prediction chart? (RSI, MACD, Moving Averages, Volume) .
- **Question 4.2:** Would you trust an AI-based prediction more if it showed a direct comparison with traditional statistical models like ARIMA? (Yes, No, Maybe) .

A.3 Survey Screenshots

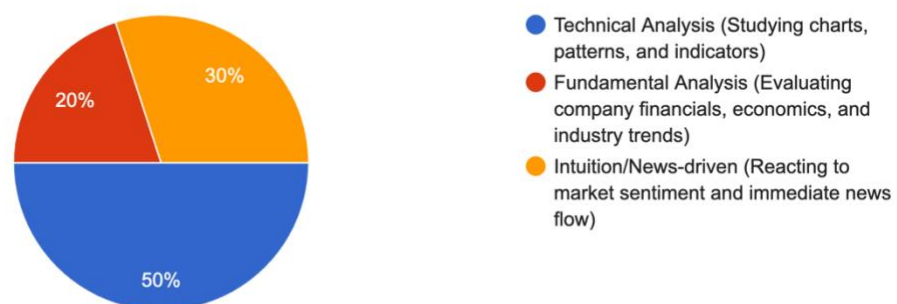
Q1: What is your current experience level in stock market trading and investment?

10 responses



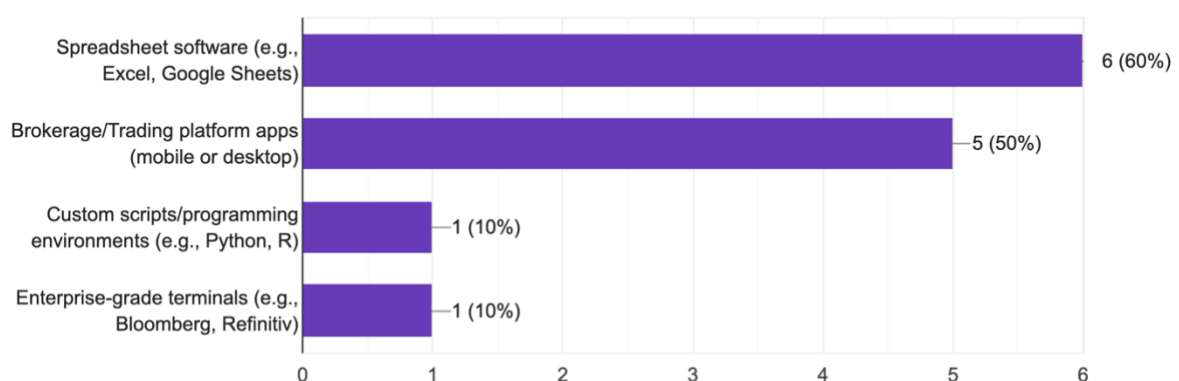
Q2: Which primary method do you rely on when analyzing potential stock investments?

10 responses



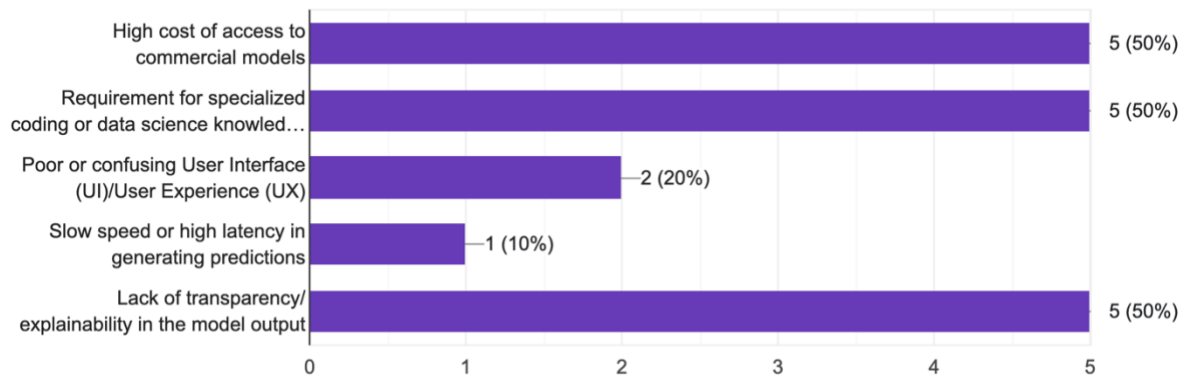
Q3: Which of the following tools do you currently use for stock analysis and tracking? (Select all that apply)

10 responses



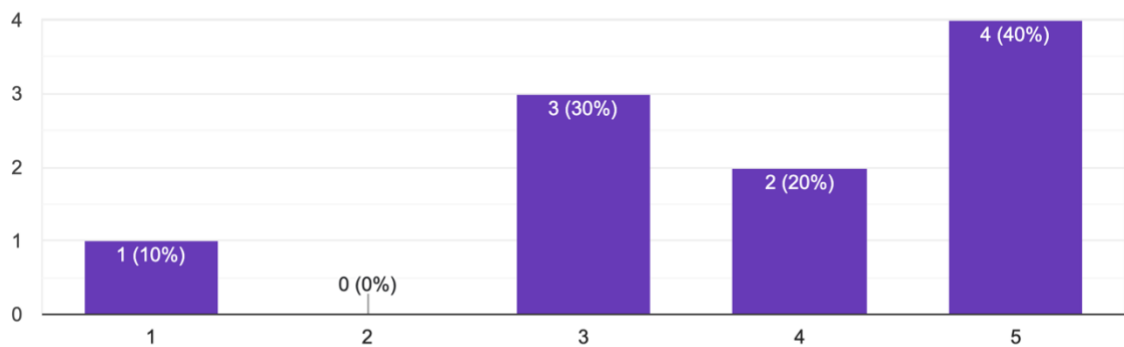
Q4: If you have previously attempted to use AI/Machine Learning models for stock prediction, what challenges did you face? (Select all relevant issues)

10 responses



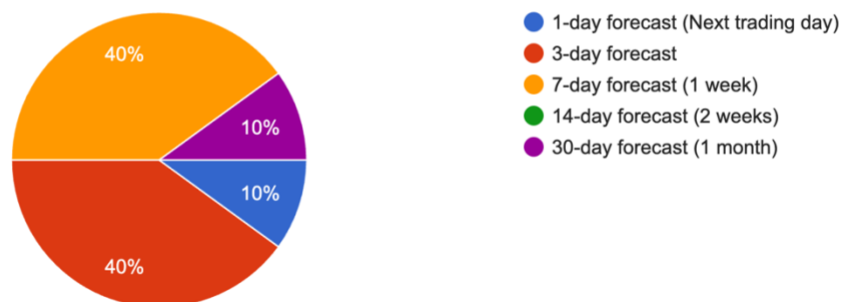
Q5: On a scale of 1 to 5, how important is it to you to access Deep Learning (specifically LSTM) stock predictions easily via a standard web browser interface?

10 responses



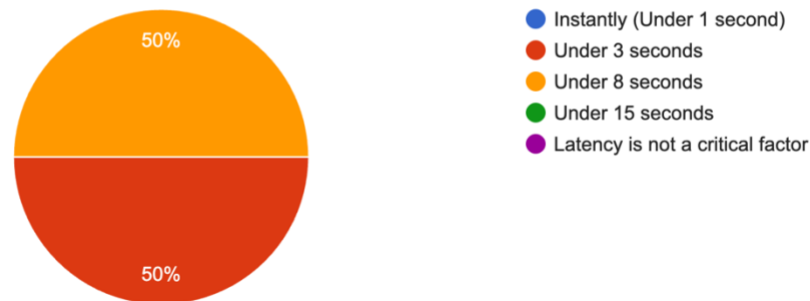
Q6: What is the most desirable prediction horizon (number of future days) you would seek from a stock prediction platform?

10 responses



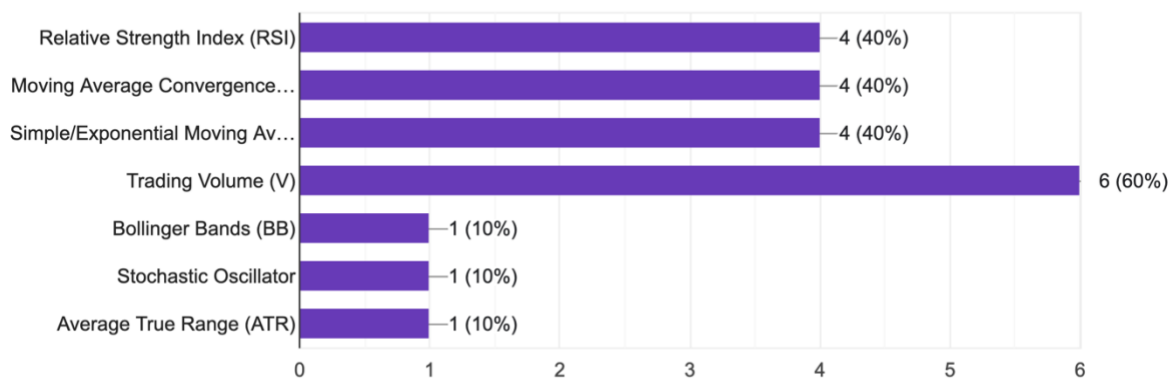
Q7: What is the maximum acceptable latency (speed) for the system to generate and display a prediction after requesting it?

10 responses



Q8: Which technical indicators do you value most highly when displayed alongside prediction data? (Select up to three)

10 responses



Q9: How would you describe your current trust level in Deep Learning (AI) models like LSTM for stock prediction compared to traditional statistical models like ARIMA?

